

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

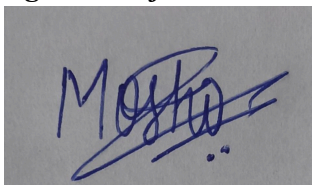
Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika M', is written over a grey rectangular background.

Annexure A

Short Answer Test

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.

ANSWER:

Input: Dataset stored in a file

Output: Cleaned and preprocessed dataset

Algorithm:

1. Start.
2. Read the dataset from the input file into a DataFrame **DF**.
3. Display the number of rows, columns, and data types.
4. Identify numerical columns and categorical columns.
5. Check each column for missing values.
6. For numerical columns:
 - a. Calculate the median of valid values.
 - b. Replace missing values with the median.
7. For categorical columns:
 - a. Find the most frequent valid category.
 - b. Replace missing values with the mode.
8. Identify and remove duplicate rows from the DataFrame.
9. Set an appropriate index for efficient row identification and access.
10. Define valid ranges for numerical attributes.
11. Filter the dataset using Boolean conditions to remove invalid values.
12. Identify invalid categorical values and replace them with their standardized forms.
13. Remove unnecessary spaces and convert categorical text to a common format, such as lowercase.
14. Normalize numerical columns using Min-Max normalization:

$$\text{Normalized Value} = (\text{Value} - \text{Minimum}) / (\text{Maximum} - \text{Minimum})$$

15. Store the normalized values back into the DataFrame.
16. Check again for missing values, duplicates, and invalid entries.
17. Verify that all numerical values are within the required normalized range **[0, 1]**.
18. Verify that categorical values belong to the predefined valid categories.
19. Save the cleaned DataFrame into an output file.
20. Display the final number of records and columns.
21. Stop.

2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.

ANSWER:

Input: Large numerical dataset

Output: Mean, median, variance, and standard deviation

Algorithm:

1. Start.
2. Read the dataset into an array A .
3. Check whether the array exists and contains valid numerical data.
4. If the array is empty, display "No data available" and stop.
5. Identify missing, non-numerical, NaN, and infinite values.
6. Filter out invalid values using array-based Boolean indexing.
7. Count the remaining valid observations as N .
8. If $N = 0$, display "No valid observations" and stop.
9. Calculate the sum of all valid values using an array aggregation operation.
10. Calculate the mean:

$$\text{Mean} = \text{Sum}(A) / N$$

11. Sort the valid array in ascending order.
12. Calculate the median:
 - If N is odd, select the middle value.
 - If N is even, calculate the average of the two middle values.
13. Calculate the deviation of every value from the mean.
14. Square each deviation and calculate their average to obtain variance:

$$\text{Variance} = \text{Sum}((A - \text{Mean})^2) / N$$

15. Calculate standard deviation:

$$\text{Standard Deviation} = \sqrt{\text{Variance}}$$

16. Detect extreme outliers using a suitable method such as the IQR rule or standard deviation threshold.
17. If required, filter extreme outliers and repeat the statistical calculations.
18. Compare the results before and after outlier handling.
19. Validate that the final statistical values are finite and meaningful.
20. Display mean, median, variance, and standard deviation.
21. Stop.

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.

ANSWER:

Input: Two or more related DataFrames

Output: Combined and validated DataFrame

Algorithm:

1. Start.
2. Read DF1, DF2, and other required DataFrames.
3. Examine the shape, columns, index, and data types of each DataFrame.
4. Identify the common key available in the DataFrames, such as Customer_ID.
5. Check whether the common key has the same data type in all DataFrames.
6. If necessary, convert the key columns to a common data type.
7. Align the indices when index-based operations are required.
8. Check whether the key contains missing values.
9. Handle or remove records with invalid key values.
10. Merge the DataFrames using the common key.
11. Select the appropriate merge type such as inner, left, right, or outer join.
12. Check the merged DataFrame for missing values created by unmatched records.
13. Remove duplicate records based on the common key.
14. Group the combined data according to required attributes.
15. Apply aggregation functions such as sum, mean, count, or max.
16. Sort the grouped or merged data according to the required column.
17. Validate the final dataset by checking:
 - a. Number of records.
 - b. Number of unique keys.
 - c. Missing values.
 - d. Duplicate records.
 - e. Data type consistency.
18. Compare the record counts with the expected result.
19. If validation fails, identify and correct inconsistent records.
20. Return the final combined DataFrame.
21. Stop.

4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

ANSWER:

Input: Large DataFrame and filtering conditions

Output: Filtered and sorted DataFrame

Algorithm:

1. Start.
2. Load the dataset into DataFrame `DF`.
3. Inspect the columns and identify the fields required for filtering.
4. Use indexing to select only the required columns.
5. Define numerical constraints, such as:
 - a. `Age >= 18`
 - b. `Age <= 60`
 - c. `Salary > 30000`
6. Define categorical constraints, such as:
 - a. `Department = "IT"`
 - b. `Status = "Active"`
7. Combine the conditions using logical operators `AND`, `OR`, and `NOT`.
8. Apply Boolean indexing to filter the DataFrame.
9. Remove rows containing missing or invalid values in required fields.
10. Verify that every remaining record satisfies the specified conditions.
11. Sort the filtered DataFrame using the required column.
12. Select ascending or descending order depending on the requirement.
13. For large datasets, perform filtering before sorting to reduce the number of records that need to be sorted.
14. Select only the required columns after filtering to reduce memory usage.
15. Use vectorized DataFrame operations instead of row-by-row processing for better performance.
16. Check the final result for duplicate or invalid records.
17. Verify that all filtering constraints are satisfied.
18. Return or display the sorted filtered dataset.
19. Stop.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand